Sustavi baza podataka

Predavanja

6. Sustav za obnovu

(1. dio)

travanj 2026.

Pogreške u SUBP-u

- različiti uzroci pogrešaka:
  - pogreške računalne opreme (softverske, hardverske)
  - prekid napajanja
  - požar, potres, eksplozija, sabotaža
  - itd.
- svaki od tih događaja može dovesti do gubitka podataka

- takve događaje nije moguće spriječiti korištenjem "besprijekornih" komponenti i "besprijekornim" dizajnom sustava → previsoka cijena

Rješenje:

- korištenje sustava za obnovu (*recovery system*)

Zadatak sustava za obnovu

- u slučaju pogreške dovesti SUBP u stanje ekvivalentno zadnjem konzistentnom stanju sustava prije pogreške. Pri tome je važno:

1. ostvariti čim veću sposobnost oporavka (*resilience*) SUBP-a
   - sustav treba biti u stanju oporaviti se nakon različitih vrsta pogrešaka
2. ostvariti čim veću raspoloživost (*availability*) SUBP-a minimiziranjem vremena potrebnog za oporavak sustava nakon pogreške
   - omjer vremena tijekom kojeg je sustav sposoban obavljati predviđenu funkciju i ukupnog promatranog vremena koje obuhvaća i vrijeme potrebno za oporavak

$$Av = MTTF / (MTTF + MTTR)$$

MTTF — Mean Time To Failure
MTTR — Mean Time To Repair

- MTTF ovisi o pouzdanosti (*reliability*)

sustav u kvaru	raspoloživost
1 sat/dan	95.8 %
1 sat/mjesec	99.86 %
1 sat/godinu	99.9886 %

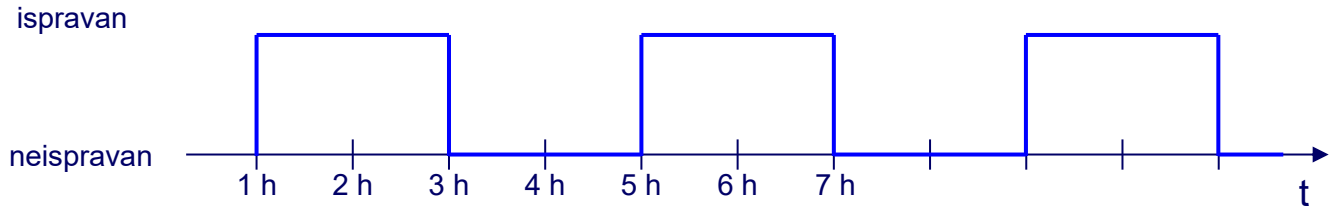
Pouzdanost (*reliability*): svojstvo sustava koje se odnosi na vjerojatnost da će sustav funkcionirati ispravno tijekom određenog vremenskog intervala.

Sustav za obnovu ne može utjecati na pouzdanost ("ne može spriječiti kvar").

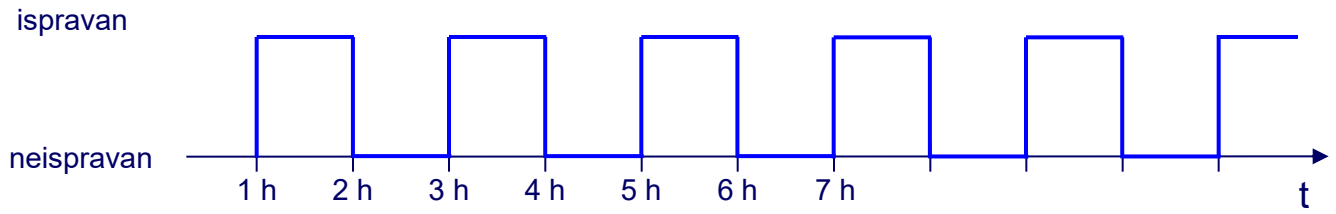
Raspoloživost, pouzdanost, srednje vrijeme oporavka

- usporediti raspoloživost, srednje vrijeme potrebno za oporavak i pouzdanost sljedećih sustava

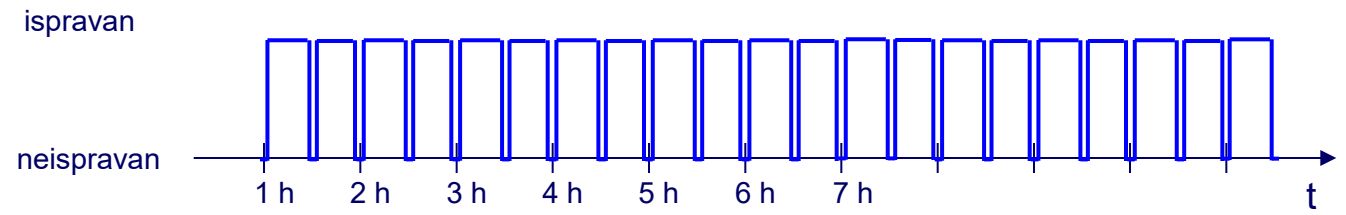
1.



2.

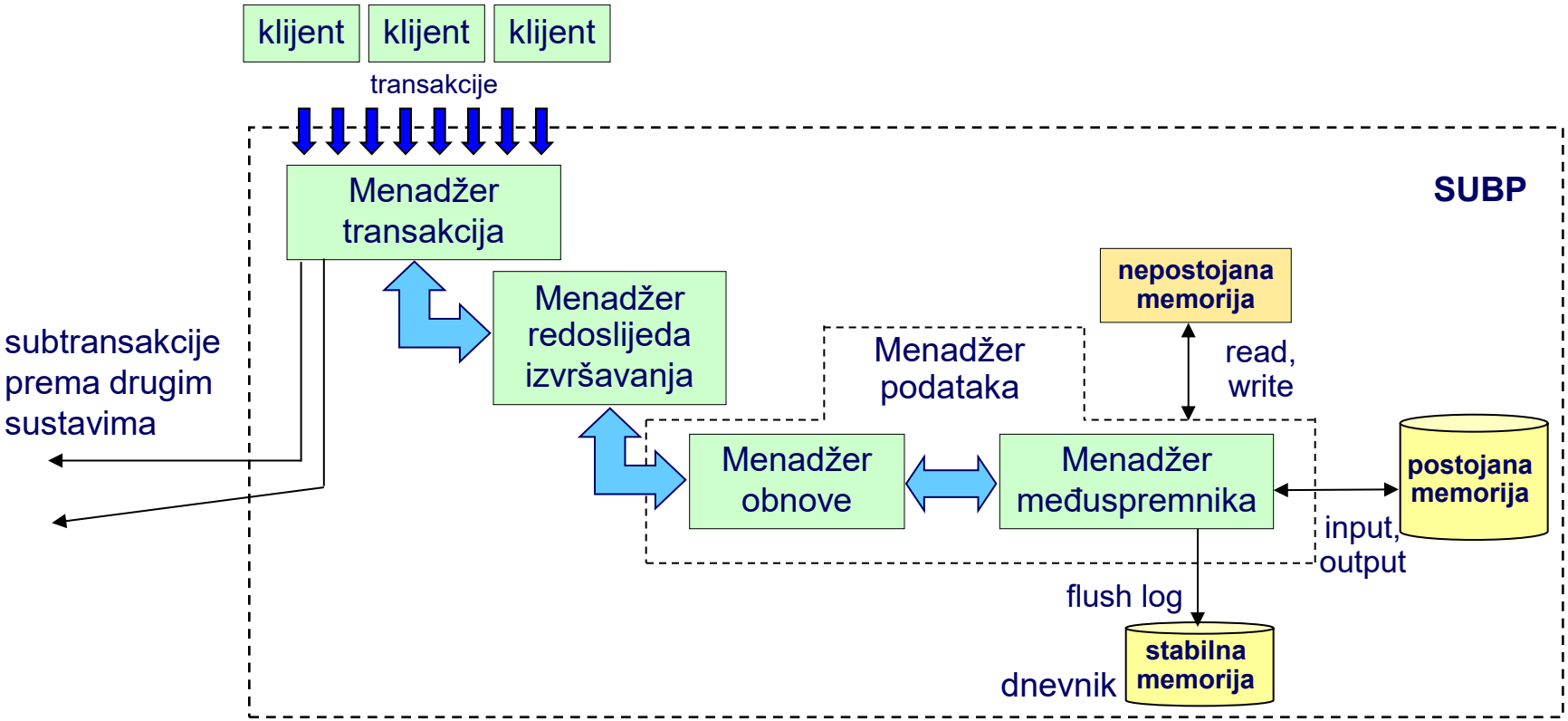


3.



Model SUBP-a

- apstraktni model sustava za upravljanje bazama podataka [Bernstein]



- na slici su prikazani samo oni dijelovi sustava za upravljanje bazama podataka koji upravljaju obnovom i istodobnim pristupom (npr. nedostaje *Query optimizer, Access Methods Manager,* itd.)

Model SUBP-a

- Menadžer transakcija (*transaction manager*) upravlja početkom i završetkom transakcije, zaprimljene operacije transakcija upućuje prema menadžeru redoslijeda izvršavanja (*scheduler*). U distribuiranim sustavima ima dodatnu zadaću usmjeravanja operacija transakcija prema drugim SUBP
- Menadžer redoslijeda izvršavanja (*scheduler*) upravlja redoslijedom obavljanja operacija transakcija: obavljanjem, odbijanjem ili odgađanjem obavljanja pojedinih operacija. Upravlja istodobnim (ili "paralelnim") pristupom
- Menadžer obnove (*recovery manager*) osigurava pouzdano potvrđivanje ili poništavanje transakcija. Podsustav je odgovoran za to da baza podataka sadrži sve efekte potvrđenih transakcija i niti jedan efekt poništenih transakcija. Između ostalog, mora biti neosjetljiv na gubitak sadržaja nepostojane (*volatile*) memorije ili na gubitak sadržaja ili kvar postojane (*non-volatile*) memorije.
- Menadžer međuspremnika (*buffer manager*) je podsustav zadužen za prebacivanje blokova memorije između međuspremnika i postojane memorije.
- Menadžer obnove i menadžer međuspremnika čine veću cjelinu koja se uobičajeno naziva menadžer podataka (*data manager*)

Uspostavljanje sustava za obnovu

- identificirati pogreške koje se u pojedinim komponentama sustava mogu dogoditi, te utjecaj pojedine vrste pogreške na sposobnost oporavka i raspoloživost sustava
- definirati postupke obnove (*recovery algorithms*) koji će osigurati otpornost sustava na različite vrste pogrešaka, odnosno omogućiti očuvanje konzistentnosti baze podataka bez obzira na vrstu pogreške

Postupci obnove

1. postupci koji se provode tijekom redovitog funkcioniranja sustava i osiguravaju prikupljanje dovoljno podataka za obnovu baze podataka nakon pogreške
2. postupci pomoću kojih se u slučaju pogreške sustav na pouzdan način vraća u konzistentno stanje

Temeljna ideja implementacije

Korištenje redundancije - svaki podatak koji je izgubljen zbog pogreške u jednom dijelu sustava, mora se moći rekonstruirati iz podataka redundantno pohranjenih u nekom drugom dijelu sustava

Vrste pogrešaka

- pogreške koje utječu na funkcioniranje SUBP-a klasificiraju se na sljedeći način:
  - pogreške transakcija (*transaction failures*)
  - pogreške sustava (*system failures, crashes*)
  - pogreške medija (*media failures*)

Pogreške transakcije

- nepredviđena pogreška koja je uzrokovana ulaskom transakcije u stanje koje onemogućuje daljnje ispravno obavljanje operacija transakcije. SUBP **implicitno** prekida i poništava dotičnu transakciju
  - pogreške koje nastaju zbog interakcije transakcije s drugim transakcijama
    - detekcija stvarnog ili potencijalnog potpunog zastoja
    - narušavanje serijalizabilnog izvršavanja
  - neočekivani prekid korisničke sjednice (npr. prekid klijentske aplikacije)

- u navedenim i sličnim slučajevima SUBP mora prekinuti i poništiti transakciju
  - poništavanje transakcije obavlja sustav za obnovu

Pogreške transakcije

- važne karakteristike pogrešaka transakcija:
  - ne dolazi do gubitka sadržaja postojane niti nepostojane memorije
  - ne dolazi do zastoja u funkcioniranju SUBP-a (transakcije koje nisu sudjelovale u pogrešci nastavljaju se izvršavati bez zastoja)

Pogreške transakcije

Primjer:

```
begin work
   read(x, p)
   read(y, q)
   p ← p / q
   write(x, q)
   write(y, p)
commit work
```



- Ako u trenutku **t** dođe do prekida korisničke sjednice, dogodit će se **pogreška transakcije**. SUBP će prekinuti i poništiti takvu transakciju.

Eksplicitno rukovanje pogreškama

- pogrešku koja se dogodi pri obavljanju neke operacije transakcije SUBP signalizira procesu koji izvršava transakciju. Time se omogućuje **eksplicitno rukovanje pogreškom**, npr. tako da se pod određenim programskim uvjetima transakcija eksplicitno prekine ili se obavi niz kompenzacijskih operacija i nastavi s izvršavanjem transakcije
- ako transakcija (tj. programski kôd transakcije) eksplicitno rukuje pogreškom, tada se pogreška **ne smatra pogreškom transakcije**, bez obzira obavi li se pri tome eksplicitno prekidanje transakcije ili se transakcija nastavi nakon obavljanja kompenzacijskih operacija
  - iako se ne radi o pogrešci transakcije, za poništavanje transakcije i ovdje će se koristiti mehanizmi sustava za obnovu (koji su to mehanizmi objašnjeno je kasnije)
  - isti mehanizam se koristi i za djelomično poništavanje transakcije: ROLLBACK TO SAVEPOINT

Eksplicitno rukovanje pogreškama

Primjer:

```
begin work
   read(x, p)
   read(y, q)
   p ← p / q
   write(x, p)
commit work
on exception (division by zero)
   p ← p / 2
end exception with continue
```

dogodilo se dijeljenje s nulom

Ako je vrijednost elementa y jednaka 0, transakcija će obaviti kompenzacijsku akciju. **Ne smatra se pogreškom transakcije.**

```
begin work
   read(x, p)
   read(y, q)
   p ← p / q
   write(x, p)
commit work
on exception (division by zero)
   rollback work
end exception
```

dogodilo se dijeljenje s nulom

Ako je vrijednost elementa y jednaka 0, transakcija će se eksplicitno prekinuti te poništiti. **Ne smatra se pogreškom transakcije.**

- iako se ne radi o pogrešci transakcije, za poništavanje transakcije i ovdje se (u primjeru na desnoj strani) koriste mehanizmi sustava za obnovu

Pogreške sustava

- SUBP dio podataka privremeno pohranjuje u nepostojanoj memoriji, u međuspremnicima baze podataka
- pogreška sklopovlja, pogreška programske podrške (operacijskog sustava ili programske podrške SUBP-a), nestanak napajanja i slično, uzrokuje trenutačni prekid rada sustava (SUBP procesa ili dretvi izvršavanja)

Pretpostavka o prekidu rada sustava u slučaju pogreške (*fail-stop* i *fail-fast assumption*) je važan mehanizam kojim se osigurava da hardverske ili softverske pogreške izazivaju trenutačno zaustavljanje sustava (*system halt*), s ciljem sprječavanja korupcije postojane memorije

- djelovanje sukladno ovoj pretpostavci omogućeno je mehanizmima samoprovjere na hardverskoj i softverskoj razini koje posjeduju današnji sustavi (*assertions*, *CPU checks*, *memory parity*, RAID)

- Važne karakteristike pogreške sustava:
  - prekid rada sustava za upravljanje bazama podataka
  - gubitak sadržaja nepostojane memorije (sadržaja međuspremnika)
  - postojana memorija je očuvana!
  - pri ponovnom pokretanju SUBP-a potrebna je obnova nakon pogr. sustava

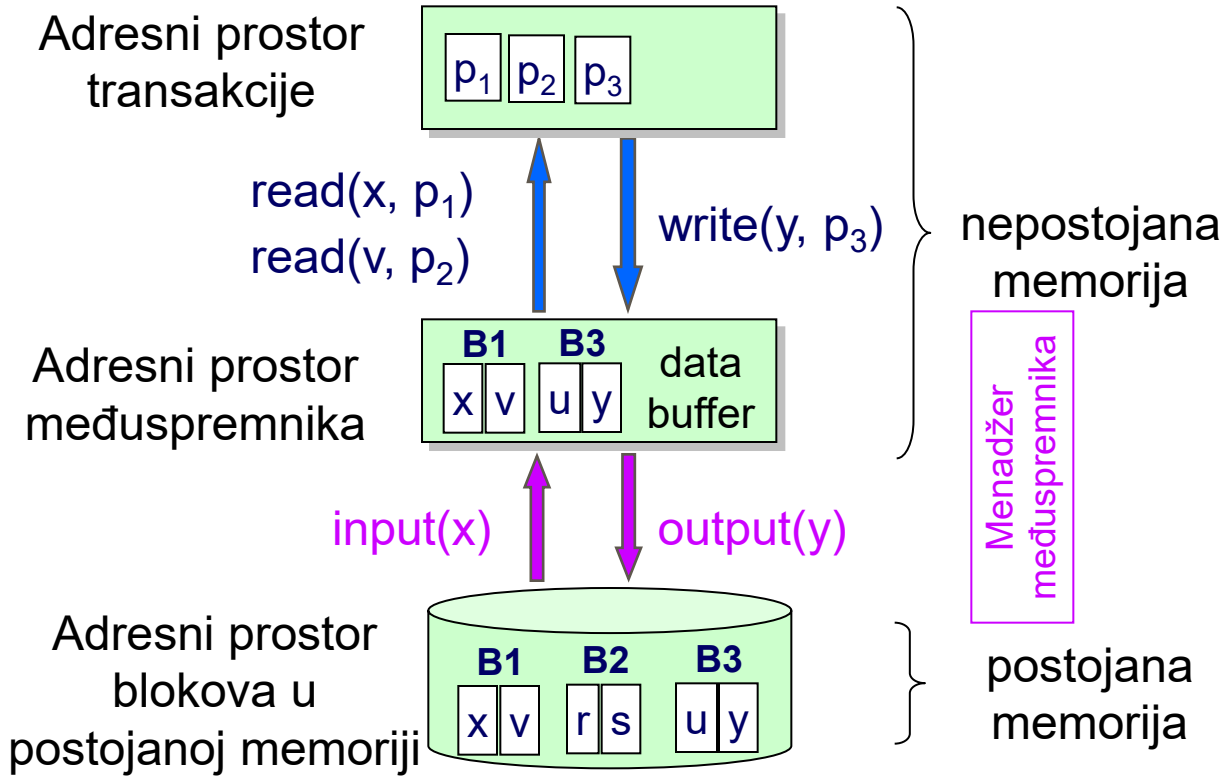
Predstavlja li kontrolirano zaustavljanje SUBP-a (*shutdown*) pogrešku sustava nakon koje je potrebna obnova?

Pogreške medija

- Pogreška medija odnosi se na djelomični ili potpuni gubitak podataka u postojanoj memoriji
  - posljedica neispravnosti sklopovlja, programske potpore, operacijskog sustava, SUBP-a
    - pogreške tijekom prijenosa podataka između nepostojane i postojane memorije
    - tipično za magnetske diskove: *disk crash, head crash*
  - požar, potres, vandalizam, teroristički napad, posljedice ratnog djelovanja
  - sabotaža, namjerno uništavanje podataka
- Važne karakteristike pogreške medija:
  - prekid rada sustava za upravljanje bazama podataka
  - gubitak sadržaja nepostojane memorije (sadržaja međuspremnika)
  - postojana memorija je djelomično ili u potpunosti izgubljena
  - pri ponovnom pokretanju SUBP-a potrebna je obnova nakon pogr. medija
- Pogreška medija u loše podešenom sustavu za obnovu u pravilu izaziva *katastrofu*
  - katastrofa (*disaster*) je tehnički pojam kojim se opisuje pogreška nakon koje nije moguća obnova

Temeljne operacije (*primitive operations*) transakcije

- SUBP koristi tri adresna prostora
  - lokalni adresni prostor transakcija
  - adresni prostor međuspremnika baze podataka
  - adresni prostor blokova na diskovima



Temeljne operacije (*primitive operations*) transakcije

Temeljne operacije

- input(x) - blok koji sadrži element x iz adresnog prostora diska kopiraj u međuspremnik
- read(x, p) - ako blok u kojem se nalazi x nije u međuspremniku, obavi input(x). Kopiraj sadržaj x iz međuspremnika u lokalnu varijablu transakcije p
- write(x, p) - ako blok koji sadrži element x nije u međuspremniku, obavi input(x). Kopiraj sadržaj lokalne varijable transakcije p u međuspremnik
- output(x) - kopiraj blok međuspremnika u kojem se nalazi element x u odgovarajući blok na disku

Temeljne operacije (*primitive operations*) transakcije

Primjer:

- dva elementa u bazi podataka (x = **3**, y = **5**)
- integritetsko ograničenje: y = x + 2
- p - lokalna varijabla transakcije T

- izvršavanje neke transakcije T se može prikazati na sljedeći način:

T	p	disk x	disk y
begin		3	5
read(x, p)	3	3	5
p ← p * 2	6	3	5
write(x, p)	6	6	5
p ← p + 2	8	6	5
write(y, p)	8	6	8
commit		6	8

- transakcija T je korektna (oba elementa uvećava za 2)
- na ovoj slici nisu prikazani sadržaji blokova međuspremnika i operacije menadžera međuspremnika (input/output)

Temeljne operacije (*primitive operations*) transakcije

- u cilju smanjenja broja U/I operacija, menadžer međuspremnika dopušta da se rezultat obavljene izmjene neko vrijeme (i uz zadovoljene određene uvjete) zadrži u nepostojanoj memoriji

T	Buffer manager	p	buff x	buff y	disk x	disk y
begin					3	5
read(x, p)	input(x)	3	3		3	5
p ← p * 2		6	3		3	5
write(x, p)		6	6		3	5
p ← p + 2		8	6		3	5
	output(x)	8	6		6	5
write(y, p)	input(y)	8	6	8	6	5
commit						
	output(y)	8	6	8	6	8

Temeljne operacije (*primitive operations*) transakcije

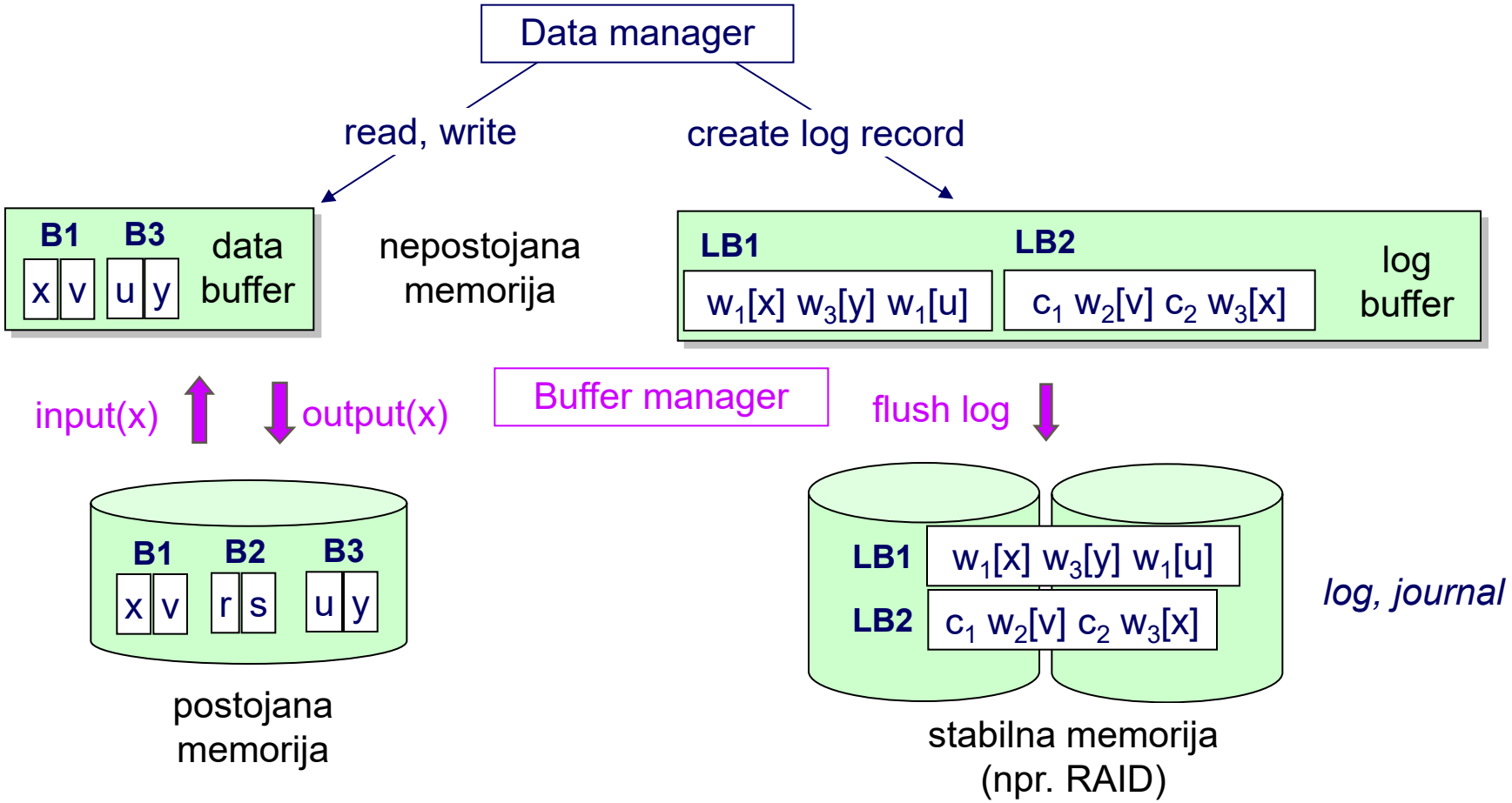
- ako se pogreška sustava ili transakcije dogodi u bilo kojem trenutku nakon obavljanja output(x), a prije obavljanja output(y), baza podataka ostaje u nekonzistentnom stanju

T	Buffer manager	p	buff x	buff y	disk x	disk y
begin					3	5
read(x, p)	input(x)	3	3		3	5
p ← p * 2		6	3		3	5
write(x, p)		6	6		3	5
p ← p + 2		8	6		3	5
	output(x)	8	6		6	5
write(y, p)	input(y)	8	6	8	6	5
commit						
	output(y)	8	6	8	6	8

Dnevnik, zapis dnevnika

- aktivnosti transakcija se bilježe u **dnevniku** (*log*, *journal*)

$w_1[x], r_2[v], w_3[y], r_3[u], w_1[u], c_1, r_3[x], w_2[v], c_2, r_3[v], w_3[x], a_3, r_4[v], w_4[x]$ ...



Dnevnik, zapis dnevnika

- aktivnosti transakcija se bilježe u **dnevniku** (*log*, *journal*)
  - zapisi dnevnika se stvaraju (*create log record*) u međuspremniku dnevnika (*log buffer*) u nepostojanoj memoriji
  - dnevnik sadrži zapise o operacijama *write*, *commit*, *abort* (ali ne i *read*) svih transakcija koje su u tijeku
    - u dnevniku se zapisi različitih transakcija međusobno isprepliću (*interleaves*) jer se upisuju redom kojim se operacije izvršavaju
- menadžer međuspremnika upravlja zapisivanjem međuspremnika podataka u postojanu memoriju (operacije *output*) i međuspremnika dnevnika u **stabilnu** memoriju (operacije *flush log*)
  - kada se točno obavljaju operacije *output*, a kada operacije *flush log*, ovisi o primijenjenoj tehnici obnove (tehnike obnove objašnjene su kasnije)
- korisno je znati da dnevnik u stabilnoj memoriji sadrži niz zapisa dnevnika (*log records*) u obliku datoteke u koju se zapisi dodaju isključivo na začelje (*append-only file*), što znači da pri zapisivanju dnevnika u stabilnu memoriju ne dolaze do izražaja loše karakteristike magnetskih diskova (*seek time* + *rotational latency*)

Dnevnik izmjena, zapis dnevnika izmjena

- **dnevnik izmjena** (*update log*) sadrži niz **zapisa dnevnika izmjena** (*update log record*)
  - različite vrsta zapisa dnevnika izmjena
    - zapis koji opisuje početak transakcije: <start T_i>
      - započela je transakcija s identifikatorom T_i
    - zapis koji opisuje operaciju pisanja: <T_i, x_j, V_{old}, V_{new}>
      - transakcija s identifikatorom T_i promijenila je vrijednost elementa x_j iz prethodne vrijednosti V_{old} u novu vrijednost V_{new}
      - zapis dnevnika izmjene stvara se kao posljedica operacije *write* (a ne operacije *output*)
    - zapis koji opisuje potvrđivanje transakcije: <commit T_i>
    - zapis koji opisuje prekid transakcije: <abort T_i>

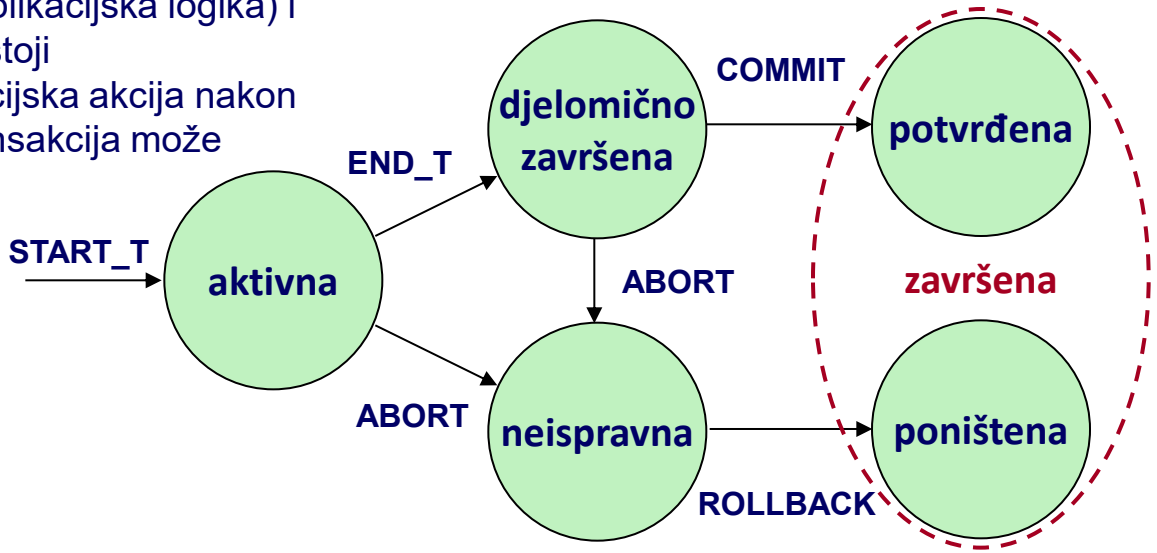
Dijagram stanja transakcije

- sa stanovišta SUBP-a, transakcija se u svakom trenutku nalazi u jednom od stanja prikazanih na slici

aktivna (*active*): operacije se obavljaju bez pogreške ili se dešavaju samo one pogreške kojima transakcija eksplicitno upravlja (aplikacijska logika) i za koje postoji kompenzacijska akcija nakon koje se transakcija može nastaviti.

djelomično završena (*partially committed*): sve operacije osim naredbe za potvrđivanje transakcije su obavljene

potvrđena (*committed*): svi efekti transakcije su **trajno** zabilježeni u bazi podataka



završena (*completed*): u stanju potvrđena ili poništena

neispravna (*failed*): eksplicitno ili implicitno prekinuta (ali još uvijek nije poništena)

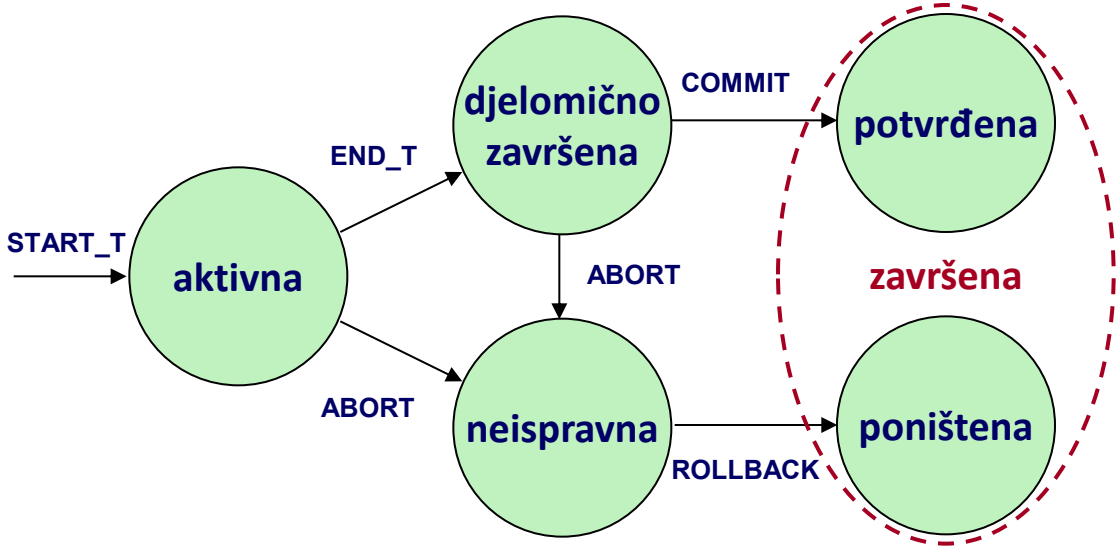
poništena (*terminated*): svi efekti operacija koje su obavljene u okviru transakcije su poništeni

Dijagram stanja transakcije

- prekid transakcije (*abort*) - korisnik ili SUBP je prekinuo transakciju
- poništavanje transakcije (*rollback*) - postupak kojim sustav poništava efekte operacija koje je transakcija eventualno provela prije prekida

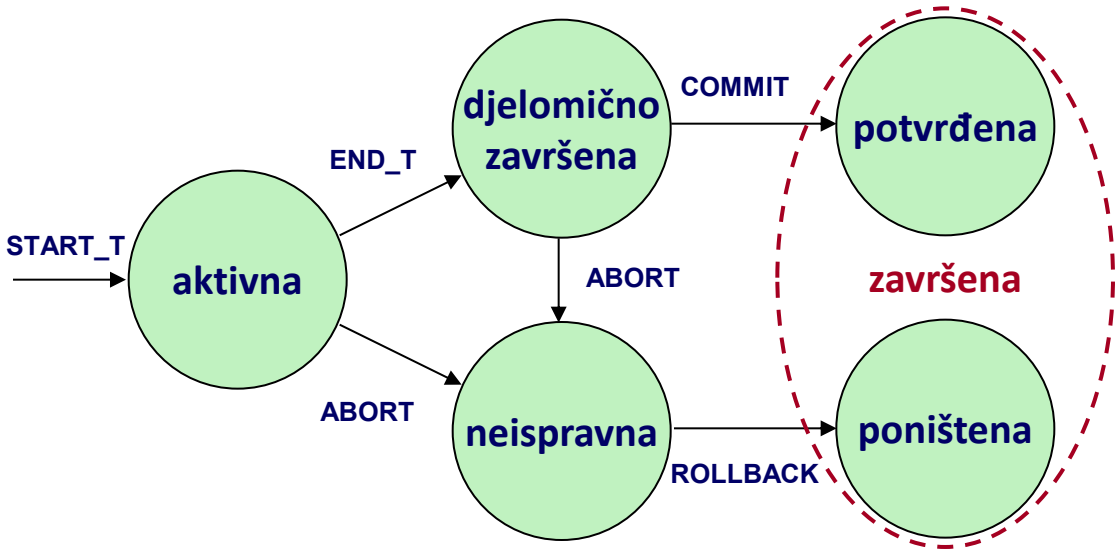
- pomoću END TRANSACTION (na slici END_T) sustavu se signalizira da su sve *read* i *write* operacije transakcije završene
- provjere integritetskih ograničenja (ako se koristi odgođena provjera integritetskih ograničenja) obavljaju se za vrijeme dok je transakcija u stanju "djelomično završena" (dakle, nakon operacije END TRANSACTION)
- najčešće se ne koriste posebne naredbe END TRANSACTION i ABORT, odnosno END TRANSACTION i COMMIT
  - SQL naredbom ROLLBACK transakcija se i prekida i poništava
  - SQL naredbom COMMIT transakcija prelazi iz stanje "aktivna" u stanje "djelomično završena", a zatim (ako za potvrđivanje transakcije ne postoji prepreka) u stanje "potvrđena"

Završena (kompletna) i nekompletna transakcija



- završena transakcija (*completed transaction*) - u dnevniku je zabilježen ishod transakcije (*commit* ili *abort*). Pri tome je irelevantno jesu li blokovi **podataka** izmijenjeni tijekom transakcije također pohranjeni u postojanu memoriju
- nekompletna transakcija (*incomplete transaction*) - u dnevniku ne postoji niti zapis *abort*, niti zapis *commit*. Pri tome je irelevantno jesu li blokovi **podataka** izmijenjeni tijekom transakcije pohranjeni u postojanu memoriju.

Točka potvrđivanja



- točka potvrđivanja (*commit point*) - transakcija T_i je dosegla točku potvrđivanja u trenutku kada je u stabilnu memoriju upisan zapis dnevnika <commit T_i>. Sve izmjene koje je transakcija načinila prije točke potvrđivanja mogu se smatrati provizornim ili tentativnim (*tentative*)
- ako se poštuje svojstvo izdržljivosti transakcija, efekti transakcije koja je dosegla točku potvrđivanja niti u kojem slučaju ne bi smjeli biti izgubljeni (bez obzira na vrstu kvara)

Korištenje dnevnika za obnovu nakon pogreške sustava

- u općem slučaju, tijekom postupka obnove nakon pogreške sustava
  - neke operacije nekih transakcija treba ponovo obaviti
  - neke operacije nekih transakcija treba poništiti

T	Buffer manager	p	buff x	buff y	disk x	disk y
begin					3	5
read(x, p)	input(x)	3	3		3	5
p ← p * 2		6	3		3	5
write(x, p)		6	6		3	5
p ← p + 2		8	6		3	5
	output(x)	8	6		6	5
write(y, p)	input(y)	8	6	8	6	5
commit						
	output(y)	8	6	8	6	8

- ako se pogreška sustava dogodila prije točke potvrđivanja, efekte transakcije T treba pomoću zapisa iz dnevnika poništiti
- ako se pogreška dogodila nakon točke potvrđivanja, pomoću zapisa iz dnevnika osigurati da su svi efekti transakcije T sigurno upisani u postojanu memoriju

- mehanizam poništavanja efekata transakcije pomoću dnevnika također se koristi **na jednak način**:
  - u slučaju implicitnog ili eksplicitnog poništavanja transakcije
  - tijekom obnove nakon pogreške medija

Korištenje dnevnika za obnovu nakon pogreške medija

- u općem slučaju, tijekom postupka obnove nakon pogreške medija
  - obnoviti bazu podataka pomoću arhivske kopije
    - sustav se vraća u stanje u kojem je bio u trenutku početka izrade arhivske kopije
  - korištenjem dnevnika izmjena obaviti operacije transakcija koje su potvrđene nakon trenutka u kojem je započela izrada arhivske kopije, te time bazu podataka dovesti u zadnje konzistentno stanje u kojem je bila prije pogreške medija

Tehnike obnove nakon pogreške sustava

- postoje različite tehnike obnove. O primijenjenoj tehnici ovisit će:
  - što sadrži zapis dnevnika izmjene
  - redoslijed obavljanja operacija *write*, *output* i *flush log*
  - postupak obnove

- *undo (undo/no-redo)*
  - tijekom obnove, operacije nekih transakcija se poništavaju
- *redo (no-undo/redo)*
  - tijekom obnove, operacije nekih transakcija se ponovno obavljaju
- *undo/redo*
  - tijekom obnove, neke se poništavaju, neke ponovno obavljaju
  - najčešće korištena tehnika u današnjim SUBP
- *no-undo/no-redo* ili *shadow paging* - neće se razmatrati
  - bez dnevnika: koriste se kopije blokova s podacima (stari blok, novi blok, *current directory*, *shadow directory*)

Undo - sadržaj dnevnika i pravila vođenja dnevnika

- zapis koji opisuje operaciju pisanja sadrži samo staru vrijednost elementa: $<T_i, x_j, V_{old}>$
  - transakcija s identifikatorom T_i promijenila je vrijednost elementa x_j čija je **prethodna** vrijednost bila V_{old} na novu vrijednost V_{new}
- pravila vođenja dnevnika:
  1. ako transakcija T obavlja operaciju write(x, V_{new}), operacija *flush log* koja će upisati zapis dnevnika $<T, x, V_{old}>$ mora biti obavljena prije operacije *output(x)*
  2. operacija *flush log* kojom se zapis dnevnika <commit T> upisuje u stabilnu memoriju obavlja se isključivo nakon što su obavljene sve *output(x)* operacije transakcije T

Pravilo koje zahtijeva da se zapisi dnevnika upisuju u stabilnu memoriju **prije** nego se u postojanu memoriju upiše promijenjena vrijednost elementa naziva se *write-ahead logging* (WAL)

- koji je stvarni sadržaj koji se upisuje u dnevnik kao vrijednost V_{old}, V_{new}?

Undo - sadržaj dnevnika i pravila vođenja dnevnika

Primjer:

- integritetska ograničenja: y = x * 2; z = x + 4;

	T_1	Buffer manager	buff x	buff y	buff z	disk x	disk y	disk z
1.	$begin_1$					1	2	5
2.	$write_1(x, 3)$		3			1	2	5
3.		flush log	3			1	2	5
4.	$write_1(y, 6)$		3	6		1	2	5
5.		output(x)	3	6		3	2	5
6.	$write_1(z, 7)$		3	6	7	3	2	5
7.		flush log	3	6	7	3	2	5
8.		output(y)	3	6	7	3	6	5
9.		output(z)	3	6	7	3	6	7
10.	$commit_1$		3	6	7	3	6	7
11.		flush log	3	6	7	3	6	7

1. ↓ 2. ↓ 4. ↓ 6. ↓ 10. ↓

log buffer: <start T_1> <T_1,x,1> <T_1,y,2> <T_1,z,5> <commit T_1>

3. ↓ 7. ↓ 11. ↓

log: <start T_1> <T_1,x,1> <T_1,y,2> <T_1,z,5> <commit T_1>

Undo - postupak obnove nakon pogreške sustava

- *restart recovery*
- za vrijeme obnove, sustav ne zaprima nove transakcije
- čitati dnevnik **od kraja prema početku** te za svaki zapis iz dnevnika
  - ako pročitaš zapis <commit T_i> dodaj T_i u listu potvrđenih transakcija
  - ako pročitaš zapis <T_i, x, V>
    - ako je T_i u listi potvrđenih transakcija, činiti ništa*
    - inače ako T_i nije u listi potvrđenih transakcija, obavi operaciju *undo(x, V)*, tj. promijeni vrijednost elementa x na vrijednost V**

* ako je u dnevniku zabilježen zapis <commit T_i>, tada su sve *output* operacije transakcije T_i sigurno uspješno izvršene već prije pogreške

** ako u dnevniku nije zabilježen zapis <commit T_i>, možda su neke *output* operacije transakcije T_i obavljene, a neke nisu. Poništiti sve efekte transakcije T_i

- što ako se pogreška sustava opet dogodi za vrijeme obnove?
  - ponavljanje postupka obnove, bez obzira u kojem je trenutku prekinut, neće izazvati probleme jer je operacija *undo* idempotentna
    - undo(x, V), undo(x, V), ... ≡ undo(x, V)

Undo - postupak obnove nakon pogreške sustava

Primjer:

- neka se u prethodnom primjeru pogreška sustava dogodila nakon 5. koraka
- na početku obnove sadržaj dnevnika jest: .... <start T_1> <T_1,x,1>
  - tijekom obnove, undo(x, 1)
    - u element x, čija je vrijednost u trenutku pogreške bila 3 (posljedica operacija u koracima 2. i 5.), zapisuje se stara vrijednost elementa pronađena u zapisu dnevnika

Za vježbu odgovoriti na pitanja:

- što će se desiti ako se pogreška sustava dogodi nakon 8. koraka?
- što će se desiti ako se pogreška sustava dogodi nakon 10. koraka?
  - smije li SUBP korisniku dojaviti "Transakcija je potvrđena" nakon 10 koraka?
- što će se desiti ako se pogreška sustava dogodi nakon 11. koraka?
  - smije li SUBP korisniku dojaviti "Transakcija je potvrđena" nakon 11 koraka?
- što će se desiti ako se pogreška sustava dogodila nakon 8. koraka, a za vrijeme obnove se opet dogodila pogreška nakon obavljanja operacije *undo(y, 2)*

Undo - nedostaci

- potreba obavljanja većeg broja U/I operacija prije potvrđivanja transakcije:
  - zapis dnevnika <commit T_i> mora se u dnevnik upisati čim je prije moguće (zbog izdržljivosti transakcije). S druge strane, <commit T_i> smije biti upisan u dnevnik tek nakon što su obavljene sve pripadne *output* operacije. Zbog toga je menadžer međuspremnika prisiljen najkasnije prije točke potvrđivanja transakcije obaviti sve *output* operacije koje su povezane s izmijenjenim elementima. Time se uvećava broj U/I operacija koje se moraju obaviti tijekom transakcije
  - primjer: mnogo vrlo kratkih transakcija mijenja isti element u bazi podataka. LRU bi vjerojatno odgodio obavljanje operacije output za taj element jer ga stalno koriste mnogi procesi, ali je zbog gore navedenih razloga prisiljen obaviti operaciju output prije dosezanja točke potvrđivanje svake od transakcija
- ograničena mogućnost obnove baze podataka pomoću arhivske kopije
  - u slučaju pogreške medija (baza podataka je uništena) koristi se arhivska kopija. Dnevnik nije upotrebljiv za ponovno obavljanje transakcija koje su izvršene nakon posljednjeg arhiviranja baze

Redo

Redo - sadržaj dnevnika

- zapis koji opisuje operaciju pisanja sadrži samo novu vrijednost elementa: $<T_i, x_j, V_{new}>$
  - transakcija s identifikatorom T_i promijenila je vrijednost elementa x_j na **novu** vrijednost V_{new}

Redo - pravila vođenja dnevnika

1. bilo koja operacija *output(x)* transakcije T smije se obaviti tek nakon što su **svi** zapisi dnevnika $<T, x, V_{new}>$ **i zapis** <commit T> transakcije upisani u stabilnu memoriju

Redo

Redo - postupak obnove

- identificirati potvrđene transakcije (za koje u dnevniku postoji zapis <commit>)
- čitati dnevnik **od početka prema kraju**
  - za svaki zapis $<T_i, x_j, V>$
    - ako je T_i potvrđena, obaviti operaciju *redo*(x_j, *V*)
    - ako T_i nije potvrđena, činiti ništa*

* ako u dnevniku ne postoji zapis $<commit\ T_i>$ tada niti jedna output(x) operacija transakcije T_i sigurno nije obavljena

Redo - pogreška sustava tijekom obnove

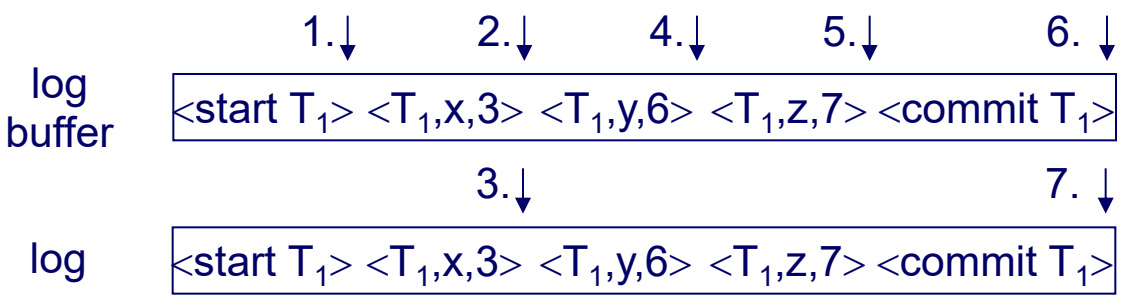
- ponovno obavljanje opisanog postupka obnove, bez obzira u kojem je trenutku prekinut, neće izazvati probleme jer je operacija *redo* idempotentna
  - *redo(x, V)*, *redo(x, V)*, ... ≡ *redo(x, V)*

Redo - sadržaj dnevnika i pravila vođenja dnevnika

Primjer:

- integritetska ograničenja: y = x * 2; z = x + 4;

	T_1	Buffer manager	buff x	buff y	buff z	disk x	disk y	disk z
1.	$begin_1$					1	2	5
2.	$write_1(x, 3)$		3			1	2	5
3.		flush log	3			1	2	5
4.	$write_1(y, 6)$		3	6		1	2	5
5.	$write_1(z, 7)$		3	6	7	1	2	5
6.	$commit_1$		3	6	7	1	2	5
7.		flush log	3	6	7	1	2	5
8.		output(x)	3	6	7	3	2	5
9.		output(y)	3	6	7	3	6	5
10.		output(z)	3	6	7	3	6	7



- *output* niti jednog elementa se ne smije obaviti prije nego je commit zapis dnevnika za transakciju zapisan u stabilnu memoriju

Redo - postupak obnove nakon pogreške sustava

Primjer:

- neka se u prethodnom primjeru pogreška sustava dogodila nakon 4. koraka
- na početku obnove sadržaj dnevnika jest:

<start T_1> <T_1,x,3>

- u dnevniku nema zapisa <commit T_1> $\Rightarrow$ činiti ništa

Primjer:

- neka se u prethodnom primjeru pogreška sustava dogodila nakon 8. koraka
- na početku obnove sadržaj dnevnika jest:

<start T_1> <T_1,x,3> <T_1,y,6> <T_1,z,7> <commit T_1>

- tijekom obnove u x se (ponovo) zapisuje vrijednost 3, u y se zapisuje 6, u z se zapisuje 7

Redo - prednosti i nedostaci

Nedostaci (u odnosu na *undo* tehniku):

- potencijalna potreba za velikim međuspremnikom podataka:
  - vrijednosti iz međuspremnika s podacima koje je promijenila transakcija T_i se ne smije upisati u postojanu memoriju (i time potencijalno osloboditi međuspremnik) prije nego se zapis dnevnika <commit T_i> zapiše u stabilnu memoriju. Zbog tog se pravila uvećava potrebna veličina međuspremnika
  - primjer: transakcija koja je izmijenila veliki broj elemenata i duže vrijeme ne obavi naredbu commit. LRU možda zatreba izbaciti te stranice iz međuspremnika (prije toga obaviti output), ali ne smije prije nego dotična transakcija dosegne točku potvrđivanja

Prednosti (u odnosu na *undo* tehniku):

- manja ograničenja redoslijeda obavljanja operacija
  - menadžer međuspremnika može po volji odgađati obavljanje operacija *output* i time optimizirati broj obavljenih U/I operacija
    - transakcija može biti potvrđena (i njezina izdržljivost garantirana) i onda kada nisu obavljene sve pripadne *output* operacije

Redo - prednosti i nedostaci

Prednosti (u odnosu na *undo* tehniku):

- veća mogućnost obnove baze podataka iz arhivske kopije
  - u slučaju pogreške medija (baza podataka je uništena) koristi se arhivska kopija. Obavljanjem *redo* operacija za zapise dnevnika potvrđenih transakcija baza podataka se može dovesti u stanje koje odgovara stanju neposredno prije pogreške medija

Redo tehnikom (jednako tako i *undo/redo* tehnikom prikazanom kasnije) omogućeno je odgađanje svih *output* operacija tijekom vrlo dugog vremena. U ekstremnom slučaju, SUBP bi mogao odgoditi obavljanje svih *output* operacije sve do trenutka zaustavljanja.

Zašto takvu mogućnost ipak ne bi trebalo koristiti?

Radi mogućnosti brze obnove nakon pogreške sustava. Ako bi se tijekom obnove sustava morao obaviti veliki broj *redo* operacija, obnova bi dugo trajala. Kako se rješava taj problem → kontrolna točka (u sljedećim predavanjima)

Undo/Redo

Undo/Redo - sadržaj dnevnika

- zapis koji opisuje operaciju pisanja: $<T_i, x_j, V_{old}, V_{new}>$
  - transakcija s identifikatorom T_i promijenila je vrijednost elementa x_j iz prethodne vrijednosti V_{old} u novu vrijednost V_{new}

Undo/Redo - pravila vođenja dnevnika

1. ako transakcija T obavlja operaciju write(x, V_{new}), operacija *flush log* kojom je zapis dnevnika $<T, x, V_{old}, V_{new}>$ upisan u stabilnu memoriju mora biti obavljena prije operacije output(x)

- razlika u odnosu na *redo*:
  - nije propisan redoslijed između zapisivanja $<$commit $T_i>$ u stabilnu memoriju i zapisivanja blokova s podacima u postojanu memoriju

Undo/Redo - postupak obnove nakon pogreške sustava

- čitati dnevnik **od početka prema kraju**
  - ako za transakciju T_i u dnevniku postoji zapis <commit T_i> tada obaviti operaciju *redo*(x_j, V_{new}) za svaki zapis <T_i, x_j, V_{old}, V_{new}> iz dnevnika*
- čitati dnevnik **od kraja prema početku**
  - ako za transakciju T_i u dnevniku ne postoji zapis <commit T_i> tada obaviti operaciju *undo*(x_j, V_{old}) za svaki zapis <T_i, x_j, V_{old}, V_{new}> iz dnevnika**

* zato jer transakcije jest potvrđena, a neke od njenih *output* operacije možda nisu obavljene

** zato jer transakcija nije potvrđena, a neke od njenih *output* operacija možda jesu obavljene

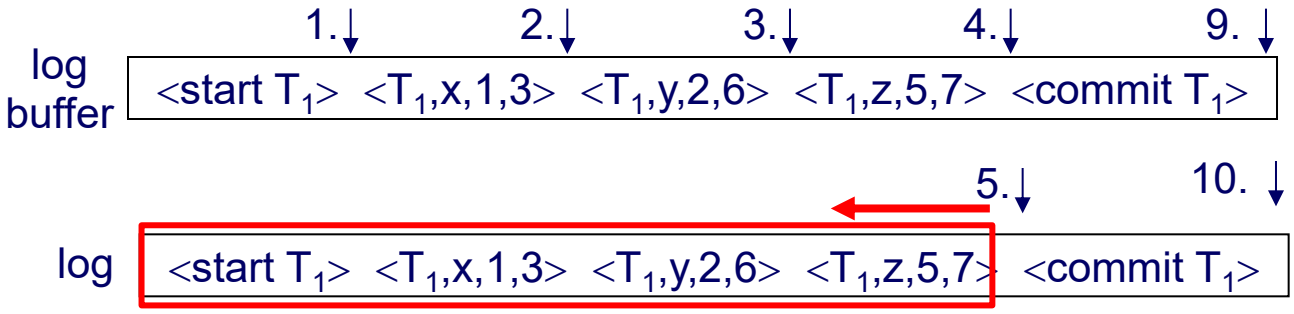
- što ako se pogreška sustava opet dogodi za vrijeme obnove?
  - ponavljanje opisanog postupka obnove, bez obzira u kojem je trenutku prekinut, neće izazvati probleme jer su operacije *undo* i *redo* idempotentne

Undo/Redo - sadržaj dnevnika i pravila vođenja dnevnika

Primjer:

- integritetska ograničenja: y = x * 2; z = x + 4;

	T_1	Buffer manager	buff x	buff y	buff z	disk x	disk y	disk z
1.	$begin_1$					1	2	5
2.	$write_1(x, 3)$		3			1	2	5
3.	$write_1(y, 6)$		3	6		1	2	5
4.	$write_1(z, 7)$		3	6	7	1	2	5
5.		flush log	3	6	7	1	2	5
6.		output(x)	3	6	7	3	2	5
7.		output(y)	3	6	7	3	6	5
8.		output(z)	3	6	7	3	6	7
9.	$commit_1$		3	6	7	3	6	7
10.		flush log	3	6	7	3	6	7



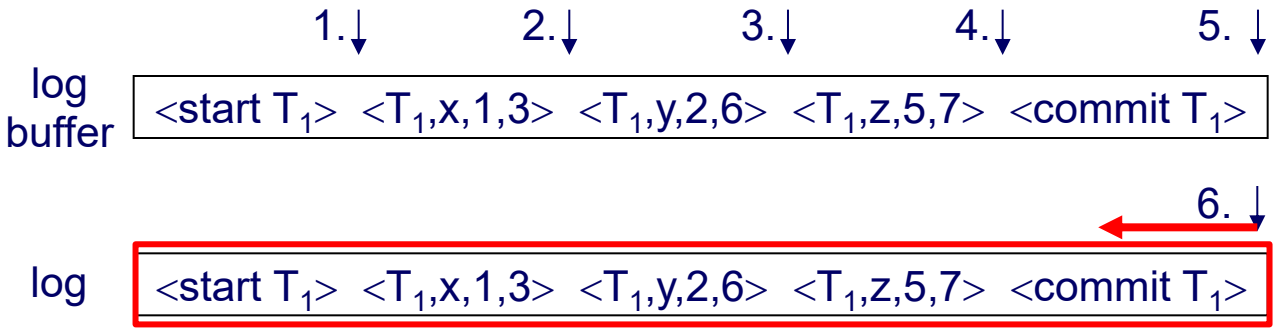
- ako se pogreška sustava dogodi nakon koraka 7. tijekom obnove obavit će se operacije *undo(z,5)*, *undo(y,2)* i *undo(x,1)*

Undo/Redo - sadržaj dnevnika i pravila vođenja dnevnika

Primjer:

- integritetska ograničenja: y = x * 2; z = x + 4;

	T_1	Buffer manager	buff x	buff y	buff z	disk x	disk y	disk z
1.	$begin_1$					1	2	5
2.	$write_1(x, 3)$		3			1	2	5
3.	$write_1(y, 6)$		3	6		1	2	5
4.	$write_1(z, 7)$		3	6	7	1	2	5
5.	$commit_1$		3	6	7	1	2	5
6.		flush log	3	6	7	1	2	5
7.		output(x)	3	6	7	3	2	5
8.		output(y)	3	6	7	3	6	5
9.		output(z)	3	6	7	3	6	7



- ako se pogreška sustava dogodi nakon koraka 7. tijekom obnove obavit će se operacije *redo(x,3)*, *redo(y,6)* i *redo(z,5)*

Undo/Redo - prednosti i nedostaci pred *redo* tehnikom

Prednost (u odnosu na *redo* tehniku):

- nema potrebe za velikim međuspremnikom podataka
  - menadžer međuspremnika može (ali ne mora) obaviti *output* operacije neke transakcije T odmah nakon upisivanja pripadnih zapisa dnevnika u stabilnu memoriju. Time što ne mora čekati završetak transakcije (kao kod *redo* tehnike) smanjuje se potreba za velikim međuspremnikom podataka

Nedostatak (u odnosu na *undo* i *redo* tehniku):

- tijekom obnove, potreban je prolazak kroz dnevnik u oba smjera
- povećani utrošak prostora za pohranu dnevnika
  - bilježe se i stare i nove vrijednosti elementa koji je promijenjen

Literatura:

- H. Garcia-Molina, J. Ullman, J. Widom. Database system implementation. Prentice-Hall International, Inc. Upper Saddle River, New Jersey. 2000.
- R. Elmasri, S. Navathe. Fundamentals of database systems, 3rd ed. Addison-Wesley Publishing Company. Reading, Massachusetts. 2000.
- A. Silberschatz, H. Korth, S. Sudarshan. Database system concepts, 6th ed. McGraw-Hill. 2010.
- P. Bernstein, V. Hadzilacos, N. Goodman. Concurrency control and recovery in database systems. Addison-Wesley Publishing Company. Reading, Massachusetts. 1987.
- Archive and Backup Guide for Informix Dynamic Server, Version 7.3, Informix Press. 1998.
- Oracle Database Backup and Recovery Advanced User's Guide 10g Release 2 (10.2)